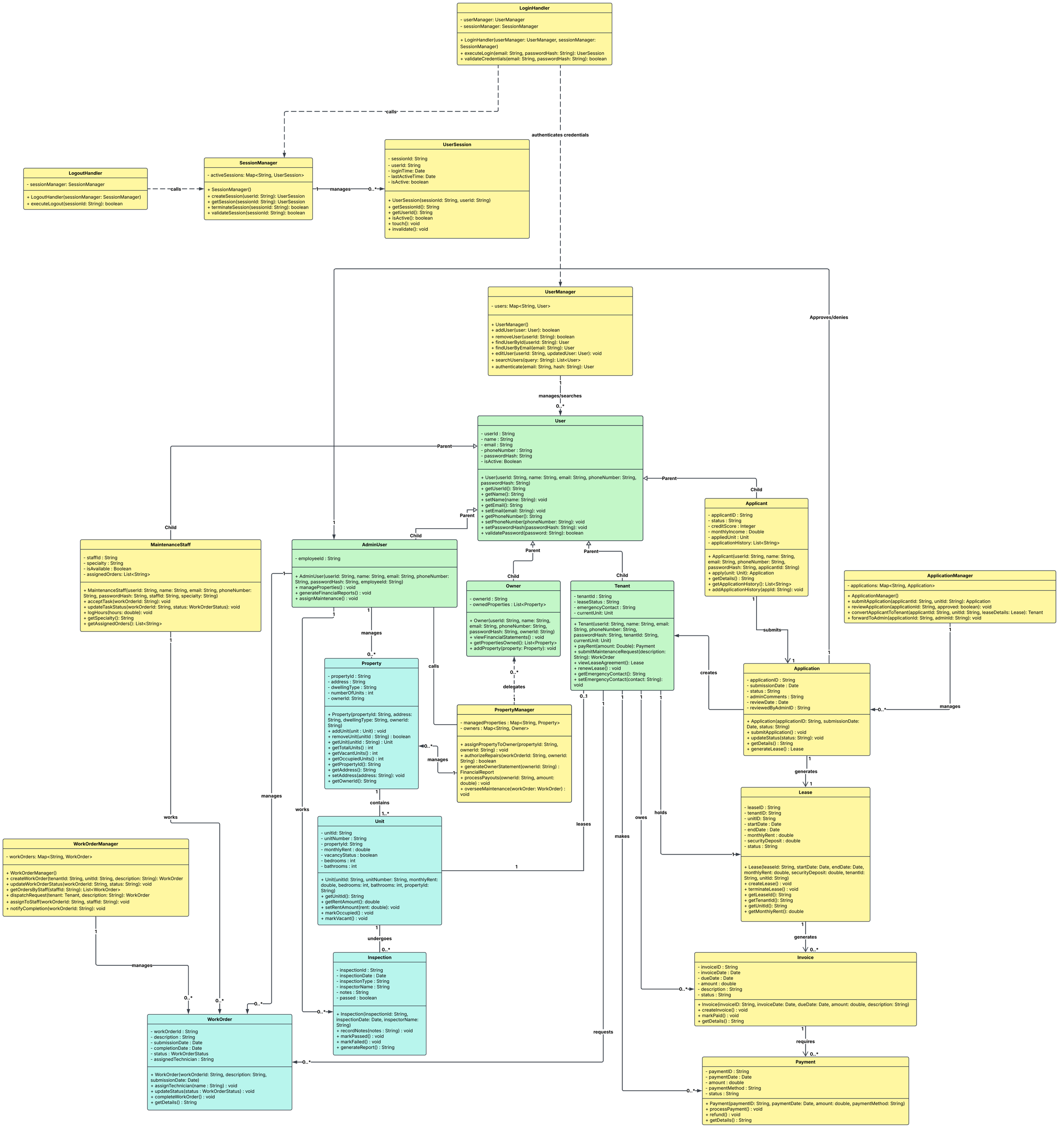

LoginHandler
- userManager: UserManager
- sessionManager: SessionManager
+ LoginHandler(userManager: UserManager, sessionManager: SessionManager)
+ executeLogin(email: String, passwordHash: String): UserSession
+ validateCredentials(email: String, passwordHash: String): boolean
calls
authenticates credentials
LogoutHandler
- sessionManager: SessionManager
+ LogoutHandler(sessionManager: SessionManager)
+ executeLogout(sessionId: String): boolean
calls
SessionManager
- activeSessions: Map<String, UserSession>
+ SessionManager()
+ createSession(userId: String): UserSession
+ getSession(sessionId: String): UserSession
+ terminateSession(sessionId: String): boolean
+ validateSession(sessionId: String): boolean
1
manages
0..*
UserSession
- sessionId: String
- userId: String
- loginTime: Date
- lastActiveTime: Date
- isActive: boolean
+ UserSession(sessionId: String, userId: String)
+ getSessionId(): String
+ getUserId(): String
+ isActive(): boolean
+ touch(): void
+ invalidate(): void
UserManager
- users: Map<String, User>
+ UserManager()
+ addUser(user: User): boolean
+ removeUser(userId: String): boolean
+ findUserById(userId: String): User
+ findUserByEmail(email: String): User
+ editUser(userId: String, updatedUser: User): void
+ searchUsers(query: String): List<User>
+ authenticate(email: String, hash: String): User
Approves/denies
1
manages/searches
0..*
User
- userId : String
- name : String
- email : String
- phoneNumber : String
- passwordHash: String
- isActive: Boolean
+ User(userId: String, name: String, email: String, phoneNumber: String, passwordHash: String)
+ getUserId(): String
+ getName(): String
+ setName(name: String): void
+ getEmail(): String
+ setEmail(email: String): void
+ getPhoneNumber(): String
+ setPhoneNumber(phoneNumber: String): void
+ setPasswordHash(passwordHash: String): void
+ validatePassword(password: String): boolean
Parent
Child
Applicant
- applicantID : String
- status : String
- creditScore : Integer
- monthlyIncome : Double
- appliedUnit : Unit
- applicationHistory: List<String>
+ Applicant(userId: String, name: String, email: String, phoneNumber: String, passwordHash: String, applicantId: String)
+ apply(unit: Unit): Application
+ getDetails() : String
+ getApplicationHistory(): List<String>
+ addApplicationHistory(appId: String): void
submits
1
MaintenanceStaff
- staffId : String
- specialty : String
- isAvailable : Boolean
- assignedOrders: List<String>
+ MaintenanceStaff(userId: String, name: String, email: String, phoneNumber: String, passwordHash: String, staffId: String, specialty: String)
+ acceptTask(workOrderId: String): void
+ updateTaskStatus(workOrderId: String, status: WorkOrderStatus): void
+ logHours(hours: double): void
+ getSpecialty(): String
+ getAssignedOrders(): List<String>
works
AdminUser
- employeeId : String
+ AdminUser(userId: String, name: String, email: String, phoneNumber: String, passwordHash: String, employeeId: String)
+ manageProperties() : void
+ generateFinancialReports() : void
+ assignMaintenance() : void
manages
Owner
- ownerId : String
- ownedProperties : List<Property>
+ Owner(userId: String, name: String, email: String, phoneNumber: String, passwordHash: String, ownerId: String)
+ viewFinancialStatements() : void
+ getPropertiesOwned(): List<Property>
+ addProperty(property: Property): void
delegates
Tenant
- tenantId : String
- leaseStatus : String
- emergencyContact : String
- currentUnit: Unit
+ Tenant(userId: String, name: String, email: String, phoneNumber: String, passwordHash: String, tenantId: String, currentUnit: Unit)
+ payRent(amount: Double): Payment
+ submitMaintenanceRequest(description: String): WorkOrder
+ viewLeaseAgreement(): Lease
+ renewLease() : void
+ getEmergencyContact(): String
+ setEmergencyContact(contact: String): void
creates
ApplicationManager
- applications: Map<String, Application>
+ ApplicationManager()
+ submitApplication(applicantId: String, unitId: String): Application
+ reviewApplication(applicationId: String, approved: boolean): void
+ convertApplicantToTenant(applicantId: String, unitId: String, leaseDetails: Lease): Tenant
+ forwardToAdmin(applicationId: String, adminId: String): void
manages
Application
- applicationID : String
- submissionDate : Date
- status : String
- adminComments : String
- reviewDate : Date
- reviewedByAdminID : String
+ Application(applicationID: String, submissionDate: Date, status: String)
+ submitApplication() : void
+ updateStatus(status: String): void
+ getDetails() : String
+ generateLease() : Lease
generates
Property
- propertyId : String
- address : String
- dwellingType : String
- numberOfUnits : int
- ownerId: String
+ Property(propertyId: String, address: String, dwellingType: String, ownerId: String)
+ addUnit(unit : Unit) : void
+ removeUnit(unitId : String) : boolean
+ getUnit(unitId : String) : Unit
+ getTotalUnits() : int
+ getVacantUnits() : int
+ getOccupiedUnits() : int
+ getPropertyId(): String
+ getAddress(): String
+ setAddress(address: String): void
+ getOwnerId(): String
calls
PropertyManager
- managedProperties : Map<String, Property>
- owners : Map<String, Owner>
+ assignPropertyToOwner(propertyId: String, ownerId: String) : void
+ authorizeRepairs(workOrderId: String, ownerId: String) : boolean
+ generateOwnerStatement(ownerId: String) : FinancialReport
+ processPayouts(ownerId: String, amount: double) : void
+ overseeMaintenance(workOrder: WorkOrder) : void
contains
1..*
Lease
- leaseID : String
- tenantID: String
- unitID: String
- startDate : Date
- endDate : Date
- monthlyRent : double
- securityDeposit : double
- status : String
+ Lease(leaseId: String, startDate: Date, endDate: Date, monthlyRent: double, securityDeposit: double, tenantId: String, unitId: String)
+ createLease() : void
+ terminateLease() : void
+ getLeaseId(): String
+ getTenantId(): String
+ getUnitId(): String
+ getMonthlyRent(): double
owes
holds
makes
leases
Unit
- unitId: String
- unitNumber : String
- propertyId: String
- monthlyRent : double
- vacancyStatus : boolean
- bedrooms : int
- bathrooms : int
+ Unit(unitId: String, unitNumber: String, monthlyRent: double, bedrooms: int, bathrooms: int, propertyId: String)
+ getUnitId(): String
+ getRentAmount(): double
+ setRentAmount(rent: double): void
+ markOccupied() : void
+ markVacant() : void
undergoes
Inspection
- inspectionId : String
- inspectionDate : Date
- inspectionType : String
- inspectorName : String
- notes : String
- passed : boolean
+ Inspection(inspectionId: String, inspectionDate: Date, inspectorName: String)
+ recordNotes(notes : String) : void
+ markPassed() : void
+ markFailed() : void
+ generateReport() : String
works
WorkOrderManager
- workOrders: Map<String, WorkOrder>
+ WorkOrderManager()
+ createWorkOrder(tenantId: String, unitId: String, description: String): WorkOrder
+ updateWorkOrderStatus(workOrderId: String, status: String): void
+ getOrdersByStaff(staffId: String): List<WorkOrder>
+ dispatchRequest(tenant: Tenant, description: String): WorkOrder
+ assignToStaff(workOrderId: String, staffId: String): void
+ notifyCompletion(workOrderId: String): void
manages
WorkOrder
- workOrderId : String
- description : String
- submissionDate : Date
- completionDate : Date
- status : WorkOrderStatus
- assignedTechnician : String
+ WorkOrder(workOrderId: String, description: String, submissionDate: Date)
+ assignTechnician(name : String) : void
+ updateStatus(status : WorkOrderStatus) : void
+ completeWorkOrder() : void
+ getDetails() : String
requests
Invoice
- invoiceID : String
- invoiceDate : Date
- dueDate : Date
- amount : double
- description : String
- status : String
+ Invoice(invoiceID: String, invoiceDate: Date, dueDate: Date, amount: double, description: String)
+ createInvoice() : void
+ markPaid() : void
+ getDetails() : String
requires
Payment
- paymentID : String
- paymentDate : Date
- amount : double
- paymentMethod : String
- status : String
+ Payment(paymentID: String, paymentDate: Date, amount: double, paymentMethod: String)
+ processPayment() : void
+ refund() : void
+ getDetails() : String